Project
bTCP: basic Transmission Control Protocol

bTCP project, Tue 10 Feb 2026, version 4.0

Deadline for FSM drawings: Fri, 2026-02-13, 23:59 (Amsterdam time). No late submissions!

Deadline basic tests output: Fri, 2026-02-20, 23:59 (Amsterdam time). No late submissions!

Final deadline: ***Tue***, 2026-04-07, 23:59 (Amsterdam time). No late submissions!

Contents

1 General remarks **2**

2 Overview **3**
- 2.1 Segment structure of bTCP . . . 4
- 2.2 bTCP connection establishment . . . 5
- 2.3 bTCP connection termination . . . 6
- 2.4 bTCP reliability . . . 6
- 2.5 bTCP error handling . . . 7
- 2.6 bTCP flow control . . . 7

3 Structure of the bTCP transport layer implementation **8**
- 3.1 The lossy layer . . . 8
- 3.2 bTCP socket interface . . . 9
- 3.3 Distinction between client- and server socket . . . 10
- 3.4 Thread safety . . . 11

4 What you are supposed to do **12**
- 4.1 Drawing the bTCP connection management finite state machines . . . 12
- 4.2 Developing your implementation . . . 13
- 4.3 Debugging . . . 14
  - 4.3.1 Logging (a.k.a `print` debugging on steroids) . . . 14
  - 4.3.2 Wireshark bTCP dissector . . . 14
- 4.4 Testing your implementation . . . 14
  - 4.4.1 File transfer on top of the bTCP protocol . . . 14
  - 4.4.2 Unit tests . . . 15
  - 4.4.3 Final words on testing . . . 15

5 Deliverables **16**

1 General remarks

Please read the following carefully:

- Work in groups of two.
- Enrol in a project group, *preferably* the one with the same group number as your assignment group if you're partnering with the same person.
- Each deliverable (python source file, drawing, pdf file, etc) should contain the names and student numbers of all group members either in the contents or the filename.
- Create a .tar.gz or zip file containing all of your deliverables.
  - Please remove `large_input.py` and correspondingly large output files from your submission!
- The name of that file should contain the student numbers of all group members (e.g. `s123456_s987654.tar.gz`).
- Submit the file via Brightspace (https://brightspace.ru.nl).
- Only *one* member of the group should submit the file, using the group submission.

You can earn up to 110 points. Point distribution is as follows:

- 10 points for the bTCP connection management finite state machines.
- 50 points for a working implementation of reliability, as demonstrated by passing the following tests. (How you can run these tests is explained in §4.4.2).
  - 10 points for handling duplication: `test_21_duplication`
  - 10 points for dealing with segments from a previous connection: `test_15_old_segments`
  - 10 points for handling corruption: `test_22_corrupted_duplicates`
  - 10 points for handling reordering: `test_30_reordering`
  - 10 points for handling packet loss: `test_60_drop_every_other`, `test_70_drop_every_other_ack`, `test_80_drop_every_other_always`
- 20 points for a working implementation of flow control, as demonstrated by passing the following tests.
  - 10 points for transferring a large file: `test_40_large`
  - 5 points for not having too many segments in flight: `test_61_window_in_flight`
  - 5 points for not overloading the server: `test_62_window`
- 10 points for *combining* both client and server socket into one class.
- 10 points for the report.
- (Up to) 10 BONUS points for including extra useful features. Make sure to document these in your report as well. Think things like extending the application to send & receive multiple files, more aggressive test cases, a fast retransmit mechanism, etc.

We *may* take away some points if you deviate too far from the instructions even though your code technically works, e.g. if you do not implement the bTCP state machine.

You *are not allowed* an NSA for this project, you *must* make a serious attempt[1].

If your code does not pass any of the basic tests `test_00_segment_length`, `test_10_connect`, or `test_11_hello_world`, then it will definitely be considered an NSA. (But passing these three tests does not guarantee not getting an NSA.)

This project will account for 20% of your final grade if and only if the grade for the project is higher than your exam grade *and* you receive a passing grade for the exam (5.5 or higher). In other words, the project grade only ever functions as a bonus. A bad but seriously attempted project does not hurt your final grade.

[1] To reiterate, because every year people are confused about this: a numeric grade between 0.1 and 5.5 is not an NSA in this sense. You only receive an NSA if we get the impression you did not put in a serious effort.

2 Overview

In this project, you will write the sending and receiving transport-layer code for a reliable data transfer protocol that we call bTCP, short for basic Transmission Control Protocol. As the name suggests, bTCP borrows a number of features from TCP:

- First, it guarantees reliable delivery of application layer data to the destination host. One of your tasks will be to define exactly how it accomplishes this.
- Second, it provides flow control (*not* congestion control!), i.e. the process of managing the rate of data transmission between sender and receiver to prevent a fast sender from overwhelming a slow receiver.
- Finally, it is connection oriented. Connections are established through a three-way handshake and shut down through a different kind of handshake.

Note that bTCP implements some of these features differently from TCP, and you have a lot of freedom to choose your particular implementation method. Unlike TCP, bTCP does not implement congestion control, as this is a complex topic and you'll already have your hands full. Also unlike TCP, bTCP does *not* implement multiplexing! Only one connection is supported, there are no port numbers. We will describe each feature in more detail in the following subsections.

As a transport layer protocol, bTCP sits between an application layer protocol and a network layer protocol:

- To the application layer, it provides the service of reliable data transfer. This service is available to applications through the bTCP **socket interface**.
- bTCP uses the (possibly unreliable) segment delivery service provided to it by the network layer.

So, the software you will be writing takes data from the application, and gives data to the application, via the bTCP socket API, i.e. the `send` and `recv` methods. It turns that data into segments, and sends them into the network layer. Rather than *use* a socket that Python provides for you, *you* are writing the socket itself.

We will provide you with a **lossy layer** that you use as the network layer and on top of which you will build your bTCP protocol. This lossy layer, in turn, uses a UDP socket to send these segments, encapsulated in UDP datagrams, over the network — we provide the code that handles that part, just be aware that you'll see UDP traffic when inspecting the bTCP flow with e.g. wireshark. The general idea is visualized in Figure 1.

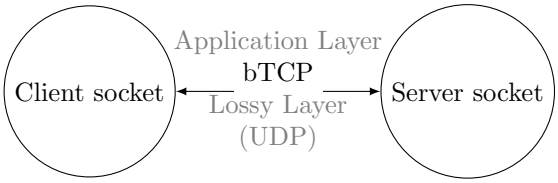

Figure 1: The bTCP stack.

In addition to writing Python 3 code[2] you will be asked to draw a few finite state machines for the bTCP connection management and write a report containing your design decisions and the justification for these decisions.

Please note that the drawing of these state machines has an **early deadline**. This is so that we can give feedback on and publish the correct state machine. If you do *not* submit this state machine in time, the points available for it will *not* be awarded even if you later submit a correct state machine in your final submission.

There is also a second **early deadline** for submitting the output of the three basic tests to demonstrate that you got started with the project. These are the same three tests that the final version of your code should (still) pass to definitely not be considered an NSA; see §5 for details.

[2] We'd prefer if you'd use Python 3.14, but any Python 3.X code with $10 \leq X \leq 14$ that runs on the standard interpreter, CPython, is acceptable.

To be able to test your socket for correctness (and grade your work), we have provided unit tests and a simple file transfer application between a client and server which uses the services provided by bTCP (see §4.4). A bTCP server socket cannot accept more than one client simultaneously, so this application is not required to accept more than one client either.

For basic bTCP operation, you *must not* alter these client and server applications, nor the existing unit tests. They already call the right socket methods, you just need to make those socket methods behave correctly. Still, you should feel free to inspect their code.

We will go over the concrete tasks in a later section.

2.1 Segment structure of bTCP

The data stream originating from the application layer is divided into individual units of data transmission called bTCP segments. These segments are what is sent between client and server. A bTCP segment consists of a segment header of 10 bytes and a data section with a fixed size of 1008 bytes. The data section follows the header and consists of payload data received from the application layer, padded with zero-bytes if it cannot fill the entire 1008 bytes with payload data. Note that the data section may be empty, i.e. it might contain 1008 bytes of padding. Segments with an empty data section can be found, for example, during connection establishment and termination.

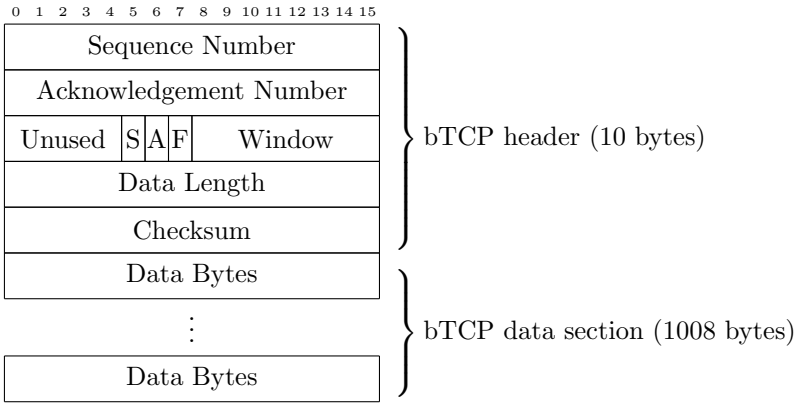

Figure 2: A bTCP segment.

The fields shown in Figure 2 have the following meaning:

- Sequence Number (16 bits): If the SYN flag is set[3], this field contains a (generally)[4] randomly chosen initial value (the 'initial sequence number', ISN) intended as the first sequence number for this new connection.

  Every new bTCP segment (in the connection) gets the next sequence number, so the second bTCP segment gets the sequence number ISN + 1, and the third one ISN + 2, and so on.

  There are two exceptions:

  1. A retransmission of an old segment must, of course, keep its original sequence number.
  2. You do not need[5] to increment the sequence number for a *trivial ACK segment*, that is, a segment that contains no data, and does not have the SYN or FIN flags set.

  By the way, the next sequence number after 65535 is, of course, 0.

- Acknowledgement Number (16 bits): If the ACK flag is set, this value is used to acknowledge received segments/data. Its value depends on how you chose to implement reliable data transfer. If the ACK flag is clear, this field has no meaning.

[3]We refer to flags as being *set* or *clear*. All of our flags consist of single bits only. Hence, we say that a flag is set if the corresponding bit is one (1) and clear if the corresponding bit is zero (0).

[4]Some tests, however, need your socket to use a very specify ISN instead of a random one. If so, this ISN is passed via the `isn` parameter of the bTCP socket constructors — the provided code already handles this.

[5]You *may* increment the sequence number for such trivial ACK segments if you must, but you will end up with an endless stream of ACKs ACKing ACKs.

- Flags (8 bits): 5 of the 8 bits are unused and set to 0. The other three bits contain the flags **S**YN, **A**CK and **F**IN. We have provided you with a reference implementation that packs them correctly into the three least significant bits of the Flags byte.
- Window (8 bits): Part of flow control, this field specifies how many segments containing data the receiver is willing to receive at the moment the segment was sent. Of course, by the time the other side receives this segment, this value might already be outdated, but based on the Acknowledgement Number (if present) a conservative estimate of the current window can still be made.
- Data Length (16 bits): Specifies the number of bytes in the data section that correspond to real payload data bytes. Even though the bTCP segment has a fixed length, we still need to be able to get the actual payload data from the segment without the padding. Without a length field it would be impossible to differentiate between "a long run of zeroes that is actual payload data" and "a long run of zeroes that is padding".
- Checksum (16 bits): The bTCP checksum, computed according to the Internet Checksum specification (refer to lecture 3, assignment 4, and assignment 5 for details). The checksum is computed over the *entire* segment with the checksum field initially set to zero. You do *not* need to add a pseudo-header with network layer addresses in front of the segment. We chose the number of bytes in the data section so that the total length is an even number of bytes, for your convenience.

2.2 bTCP connection establishment

Connection establishment in bTCP is largely the same as in TCP. Before any segments are exchanged between sender and receiver, a three-way handshake is performed.

The steps taken are the following[6]:

1. The client randomly generates a 16-bit value, say x, puts this in the Sequence Number field of a bTCP segment with the SYN flag set, and sends the segment to the server.
2. The server receives that segment. The server puts $x + 1$ in the Acknowledgement Number field of a bTCP segment with both the SYN and ACK flags set. The server then generates a random 16-bit value of its own, say y, and puts this in the Sequence Number field of that segment. The server then sends the segment to the client.
3. The client receives that segment. The client puts $y + 1$ in the Acknowledgement Number field of a bTCP segment with only the ACK flag set. The Sequence Number field of that segment is $x + 1$.
   - Just like in TCP, Acknowledgement segments without data (i.e. segments with Data Length set to 0 *and* neither SYN nor FIN set) do *not* advance the Sequence Number.
   - Therefore, the Sequence Number field of the first segment carrying *data* to the bTCP server should also be $x + 1$.
   - You *may* choose to piggy-back the data on this handshake Acknowledgement segment. In that case, the Sequence Number advances normally for the next segment.

When the client gets a response from the server to its initial SYN segment, the client should (among other things) check the Acknowledgement Number, and ignore the server's response when it's invalid. (After all, this might be a stray packet from a previous connection.)

Should the client never get a response from the server to its initial SYN segment within some specified time interval, it will simply try the first step again. The number of retries should be bounded by some specified constant. The number of tries is up to you to choose, but the total time spent on trying to connect is specified via the `timeout` paramater to the `connect` method.

The handshake accomplishes a number of things:

[6]The exact Acknowledgement Numbers and Sequence Numbers may differ depending on how you choose to implement acknowledgements. The numbers shown here are for TCP-style acknowledgements, where the number actually equals the next sequence number that the receiver *expects* to receive. Go-back-N- and selective-repeat-style acknowledgements as shown in lecture 3 and 4 might use x and y as Acknowledgement Numbers instead.

1. exchange initial sequence numbers,
2. prove that both parties can reach each other, and
3. learn the advertised window size.

We haven't shown you how to send or learn the advertised Window size here. For more information on that, read section 2.6.

Due to the size and difficulty of this project, each year many people never get past the handshake.

Correctly handshaking is actually one of the more complicated things about this protocol. Considering the distribution of points, you may want to, and are in fact encouraged to *postpone* implementing the handshake until last.

Instead of negotiating with a handshake, simply hardcode some sensible values. Make `connect` and `accept` move into the right state without sending anything, and start the server *before* the client.

2.3 bTCP connection termination

Connection termination in bTCP is similar to connection termination in TCP, but simplified. The steps taken are the following:

1. The client sends a bTCP segment with the FIN flag set.
2. The server responds with a bTCP segment with the ACK and FIN flags set.
3. The client responds to the FIN with an ACK and closes the connection.

The difference between TCP and bTCP is that in bTCP we can *always* combine the FIN and ACK of step 2, resulting in a *three*-way, rather than four-way, shutdown.

Should the client never get a response from the server to its initial FIN segment within some specified time interval, it will simply try the first step again. The number of retries should be bounded by some specified constant. If the number of retries has been exceeded and a bTCP segment with ACK and FIN flags set has still not been received, the client should just assume that the server had enough time to close its end of the connection.

Again, the timeout between retries, and number of tries are up to you to choose, but the total time that can be spent trying to sever the connection is passed to `shutdown` via the `timeout` parameter.

Similar to connection handshaking, you may want to postpone shutdown handshaking until after you can successfully transfer data uncorrupted. Initially, you can use a (long!) timeout with no segments being received at all as an indication that the client is done sending, and that is what is implemented in the provided framework.

2.4 bTCP reliability

The bTCP protocol provides reliable data transfer: data is delivered from sending process to receiving process, correctly and in order. It is up to you how it accomplishes this. At a minimum, your solution should make use of:

- checksums, to verify the integrity of segments,
- sequence numbers, to identify segments,
- acknowledgement numbers, to determine *what* segments to retransmit, and
- timers, to determine *when* to retransmit segments.

Possible solutions include Go-Back-N, Selective Repeat, and TCP. Each has its own advantages and disadvantages. You are expected to explain which one you chose, and what advantages and disadvantages made you choose it ("simplicity of implementation" *is* a valid reason).

2.5 bTCP error handling

What should be done with segments that break protocol in some way? Such as:

1. A segment with invalid checksum.
2. A segment with the SYN flag set after the connection has been established.
3. A segment that acknowledges a sequence number not known to have been recently sent.

In bTCP such segments should simply be ignored, as if they were never sent. This will trigger a retransmission of some kind, hopefully resulting in a valid segment down the road. If after some set number of retries, no (valid) reply is received, the operation (connecting, transmitting data or closing the connection) should be aborted, and the application layer notified.

Note that such retransmission causes delay and additional traffic. TCP allows a peer to immediately 'reset' a connection upon certain invalid segments (preventing this additional traffic) by sending a message with the RST flag. For simplicity, bTCP does not include this feature, our segments do not have a RST flag.

2.6 bTCP flow control

The idea behind bTCP flow control is that the receiver sends feedback to the sender about the state of its receive buffer as to not get overwhelmed by the sender, i.e. to make sure that this buffer does not overflow, because this would lead to data being discarded. To control the amount of data that can be sent, the receiver advertises a receive window, which communicates the spare room in the receive buffer. Every bTCP segment received by the receiver is (eventually) acknowledged. The receiver advertises its receive window in these acknowledgements.

During the three-way handshake, your client socket should observe the advertised window size. After connection establishment it should send as many segments as it can within the window, and wait for acknowledgement segments before sending any more segments. E.g. if the advertised window has size 100, then your client socket should send 100 data segments and wait. Once the server has acknowledged the first data segment, your client socket could send another segment. This way there should never be more than 100 segments travelling through the network at any given time.

Even so it is still possible to overwhelm the receiver if the application layer is slow to empty the receive buffer. To implement flow control properly, you should implement *updating receive windows*, where the server actively changes the Window field in its ACK segments based on the number of free spots for new data segments in its receive buffer. The client should pay attention to this receive window and postpone sending data segment accordingly. Make sure that if the window ever drops to 0, retransmissions and non-data segments are still sent.

We assume you'll be able to figure out how to do all this from the lecture recordings of lectures 3 and 4 and the lecture slides.

3 Structure of the bTCP transport layer implementation

The bTCP transport layer implementation is sandwiched between the application layer and network layer. *Each* of those layers has *independently triggered* events. A few examples of events:

- data segments can arrive on the network layer, requiring your bTCP application to inspect them for errors, reorder them, and make the ordered data available to the application.
- acknowledgement segments can arrive on the network layer, which requires no interaction with the application layer at all.
- the application may want to send some data, requiring your bTCP implementation to take that data, turn it into bTCP segments, and send them.
- a timer may expire, triggering a retransmit of unacknowledged segments.

Clearly, we are not dealing with software that has an easily defined flow through it, like a Read-Execute-Print-Loop. This is an event-driven process. In general, there are different ways of handling such an event-driven process, like asynchronous I/O. The solution we have chosen is a simple form of multi-threading:

- There is an "application thread". This is the thread of the client- and server programs, the one that would exist anyway. It is the thread in which the program calls the functions of the socket API, like `send`, `recv`, `accept`, etc.
- Then, there is a second "network thread". We explicitly create this thread in the lossy layer. This thread calls two methods in your bTCP implementation, which is clearly documented in the source code of the framework. This is the thread in which *most* of your code implementing the actual bTCP protocol will run.

3.1 The lossy layer

We simulate the network layer by building a lossy layer using UDP, which, as you should recall, is an unreliable transport layer protocol. There are two reasons for doing it like this:

1. it avoids complexity, and
2. it teaches you that the theoretical layered model as presented in both the textbook and lectures is not limited to the simple stack presented until now. We can wrap protocols in other protocols, and nothing prevents you from doing this. The data section of UDP may contain anything.

The lossy layer that is provided to you spawns its own network thread that continuously reads from the UDP socket.

Whenever a bTCP segment can be read from the socket, the lossy layer reads this segment and calls a method called `lossy_layer_segment_received`, passing the segment to it. You are supposed to write its implementation. That method should handle the incoming segment, i.e. things like checksum verification, receiving acknowledgements and registering their corresponding segment as being acknowledged, etc.

If no segment has been received for a set number of milliseconds (default 100, but you are free to change that constant), the lossy layer will call a method called `lossy_layer_tick`. Again, you are supposed to write its implementation. This allows you to do work in the network thread without having to wait for a segment to arrive. For example, you could trigger checking for timeouts on acknowledgements there.

Be careful, however: `lossy_layer_tick` will *not* be called "every 100 milliseconds", but only "every 100 milliseconds since the last segment arrived". In particular, if segments are continuously arriving 50ms apart, this method will *never* be called. So you cannot rely on counting the calls to this method to implement your timer! You will need some helper methods called from both `lossy_layer_tick` *and* `lossy_layer_segment_received` in order to ensure correct functionality if you do work in the network thread that is not directly handling received segments, which you *almost certainly will be*.

We have provided a *very* rudimentary implementation of these functions that manages to transfer a file on a *perfect* network with a fast enough receiver. It's using bTCP segments but none of the functionality (except Data Length) is implemented. It behaves like UDP: unreliable, with no transport layer error handling, or indeed acknowledgement of data reception, at all. The reason for this implementation is that many people got stuck on getting *any* segments across the network, and transferring data between application and network threads, without some demonstration of how to do so. You should build on this, adding the bTCP state machine, the buffer of unacknowledged segments for retransmission, the sending and proper handling of acknowledgements, etc. Of course, you should feel free to alter the given implementation where needed.

See the documentation in `client_socket.py` and `server_socket.py` for details about how to use all this.

3.2 bTCP socket interface

The interface between the file transfer application and bTCP is a pair of bTCP sockets, a *client socket* and *server socket*. As far as the application layer is concerned, each bTCP socket has just two states, *closed* and *established*. The client socket API consists of the following methods:

- `connect(self)` will play the client's part in connection establishment, and can be called when the socket is in the *closed* state. The `self` parameter refers to the socket class. The `connect` method should not return until the handshake has been finished as far as the client is concerned (i.e., the SYN–ACK-segment has been received).

  If after `timeout` seconds (see `__init__` below for "`timeout`") no connection has been established, the `connect` method should `raise TimeoutError()` to signal the problem to the application layer, and keep the socket in the *closed* state so that `connect` can be called again. If `connect` was successful, the socket is now in the *established* state.

- `send(self, data)` will take `data` from the application layer, encapsulate it in a number of segments, and (attempt to) put these in a *send buffer* to be sent in a reliable way to the server. Can only be called in the *established* state. The `send` method may return immediately and does not need to wait for any acknowledgement from the other side that the data has been received, or even for the data to have been sent out over lossy layer.

  The `send` method must return the number of bytes `n` of `data` that have been successfully enqueued for sending. If not everything was enqueued (because the send buffer is full) we may have `n < len(data)`. This means that the application layer must `send(data[n:])` later (calling `send` this way is already implemented in the example application, you just need to ensure `send` returns the right value).

- `shutdown(self)` will perform connection termination, and can only be called in the *established* state. It should not return until all outstanding segments have been sent and the connection has been properly terminated (i.e. a FIN–ACK has been received), all within `timeout` seconds (see `__init__` below). If not, `shutdown` should raise a `TimeoutError`. But in either case the socket should return to the *closed* state, so that `connect` can be called afterwards.

  Note that the bTCP `shutdown` described here behaves differently from the regular TCP `shutdown`, which does *not* block until the connection has been terminated. TCP clients are generally expected to wait for `recv` to return `b""`. A bTCP client, however, does not need to have a `recv` method. Hence the difference.

- `close(self)` cleans up any state. After calling `close` the socket can no longer be used.

- `__init__(window, timeout, isn=None)` creates a new client socket.

  The `window` parameter is initially only relevant for server sockets (see below).

  The `timeout` parameter specifies the time limit on `connect` and `shutdown`.

  The `isn` paramater, if not `None`, indicates what the Sequence Number of the first SYN segment should be. This is used in some unit tests.

The server socket API consists of the following methods:

- `accept(self)` can only be called when the socket is in the *closed* state, will play the server's part in connection establishment, and will not return until a connection has been established. If this does not happen within `timeout` seconds (see `__init__` below), it should raise a `TimeoutError`, and keep the socket in the *closed* state. If `accept` is successful, the socket is in the *established* state.

  Note that our bTCP API has no separation between `accept` and `listen` (i.e. there is no `listen` at all).

- `recv(self)` returns data received via bTCP, thereby delivering it to the application layer. Can be called both in the *established* and *closed* states. When no data is available yet (in the *established* state), `recv` blocks until some more data is available (or the connection is closed). Only when it is clear all data has been received, and no more is to come (that is, when the socket is in the *closed* state) does `recv` return the empty byte string `b""`.

  Note that the server API provides no way for the application layer on the server side to close the connection. Instead the connection is closed by calling `shutdown` on the *other* socket.

  The provided code does not implement the server handshake, and instead considers the connection closed after not having received any segment for `timeout` seconds.

- `close(self)` cleans up any state. After calling `close` the socket can no longer be used.

- `__init__(window, timeout, isn=None)` creates a new server socket.

  The `window` parameter indicates the exact number of incoming bTCP data segments this socket is willing to buffer for the application layer. If the application layer never calls `recv()` on this socket, and you have implemented flow control, no more than `window` data segments will ever be sent by the *other* socket over the lossy layer (excluding retransmissions).

  The `timeout` parameter specifies the time limit on `accept`.

  The `isn` paramater, if not `None`, indicates what the Sequence Number of the first SYN(–ACK) segment should be. This is used in some unit tests.

3.3 Distinction between client- and server socket

Initially, you will implement the client and server sockets in two separate classes: one for the client socket, and one for the server socket. Although the bTCP protocol is inherently symmetrical, you implement the data communication one-way and your file transfer application should be able to function using those one-way sockets.

The reason we do it this way is that for the initial implementation you may find it easier to have a clear separation: the sending side, responsible for sending payload-carrying segments and registering their acknowledgements, and the receiving side, responsible for receiving payloads and sending their acknowledgements. The `lossy_layer_segment_received` and `lossy_layer_tick` methods in both these classes will probably look quite different from each other.

You can, of course, already put (helper) methods common to both sockets in their parent class `BTCPSocket`.

Eventually however, once you have gotten these sockets working, for full credits you should combine the two sockets into a *single* socket implementation that is capable of handling *bidirectional* data transfer. The only difference between a server and a client socket then is on which one an application calls `accept` and on which one an application calls `connect`. For this, you will need to *merge* the implementations of the `lossy_layer_segment_received` and `lossy_layer_tick` methods, and alter `client_app.py` and `server_app.py` accordingly.

Back up your project (or use version control) before starting on this! That way, if you don't succeed, you still have the version with separate sockets to submit.

Of course you are allowed to work on a merged implementation from the start, but be aware that we made the conscious decision to provide you with this simpler unidirectional version for the initial implementation for a reason. The project is complex enough as it is.

3.4 Thread safety

There will obviously need to be some form of thread-safe data transfer between the application thread and network thread and back. Booleans and enums can be used to signal state changes from one thread to another[7]. You can assign the value `True` or `False`, or e.g. `bTCPStates.CLOSED` to an attribute in one thread, and read it (in a loop, or otherwise) in another thread. We do not think you will need more advanced synchronization (e.g. locks) in this project, but you may find otherwise. However, for passing around the bytes of payload data and the prepared segments, Python's `List` **is not inherently thread-safe**. To ensure that you can pass these between the two threads, you should use some instances of `Queue` — an inherently thread-safe FIFO collection — or other inherently thread-safe collections.

The rudimentary implementation we've provided uses Queues that are bounded in size: at most 1000 elements, where elements are `bytes` objects of at most 1008 bytes each. Until you get flow control working, you may find you need to make the Queue on the receiving side (much) larger to avoid losing data to Queue overflows.

Our rudimentary implementation uses these Queues to pass the *data* to the network thread. Turning the data into segments and sending these is only done in the network thread. Alternatively, you *can* send segments directly from the application thread by calling the right function of the Lossy Layer, but we do not recommend this, because:

1. Acknowledgements and retransmissions will need to happen in the network thread, so you will still need to pass the segments to the network thread anyway.

2. The more data is mutated by multiple threads, the more complicated the bugs.

It's probably better to send signals to the network thread, e.g. `start_connection = True`, and have the network thread act upon them accordingly.

[7] At least, on the standard CPython interpreter, see https://docs.python.org/3/faq/library.html#what-kinds-of-global-value-mutation-are-thread-safe.

4 What you are supposed to do

We have provided you with a number of Python 3 files which contain the emulated network layer, i.e. the lossy layer, and the stubs for any procedures that you should write, with some already having a rudimentary implementation to transfer a file over a perfect network. These procedures are by no means exhaustive, i.e. you should feel free to add any helper methods you need.

In addition to this, you will find a file transfer application and unit tests, which will be described shortly. The flow which the program follows can be seen in Figure 3.

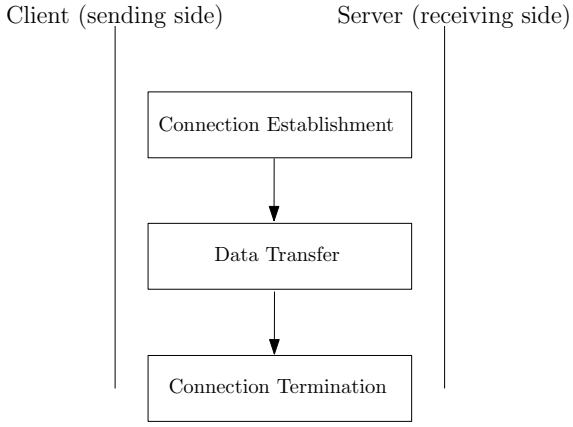

Figure 3: The different stages of bTCP

The rudimentary implementation already *calls* the requisite methods, even those not yet implemented.

Your tasks are the following:

1. Draw and submit the bTCP connection management finite state machines for both client, server, and a combined version for the combined socket. See the details in section 4.1. **Note that there is an early deadline for this!**

2. Read through this document and the three `btcp/{client,server,btcp}_socket.py` source files provided. Make the necessary modifications so that the three basic tests run via `python3 unittests.py -l DEBUG -v -k test_00 -k test_10 -k test_11` without crashing, and submit the output as proof you did this. **Note that there is another early deadline for this!**

3. Think about how you want to implement the reliable data transfer and make a choice.

4. Implement the bTCP protocol using your solution for reliable data transfer and flow control.

5. Use the unit tests and the file transfer application, described below, to test your implementation. Do not think of the unit tests as something that you only use once you believe you have finalized everything. Testing and developing go hand in hand and happen concurrently.

6. Write a report documenting all design decisions and the justification for these decisions. Again, it is best if you write while you develop as opposed to writing the report at the end when you have forgotten about all the details. Writing things down also makes you reason about what you are doing.

4.1 Drawing the bTCP connection management finite state machines

You should draw and submit three finite state machines (FSMs), according to the format we have been using in the course, for bTCP:

1. One FSM for the progression of connection states for the *server*.

2. One FSM for the progression of connection states for the *client*.

3. One FSM that *combines* the two, merging states that are conceptually the same.

Make sure you include the transition labels, i.e. the events that trigger a transition between states and the actions that are taken in that transition.

To understand what we mean by all this, figures 4 and 5 show the connection management FSMs for a TCP server and client, as shown in lecture 4. The states that would be merged in a combined FSM would be `CLOSED` and `ESTABLISHED`. The merged states have all the transitions coming in and going out of the individual states of the separate FSMs.

Of course, bTCP has a different state machine, so don't just copy these (although it *will* look similar to these). We have provided an incomplete list of possible states in `btcp_socket.py` as an enum.

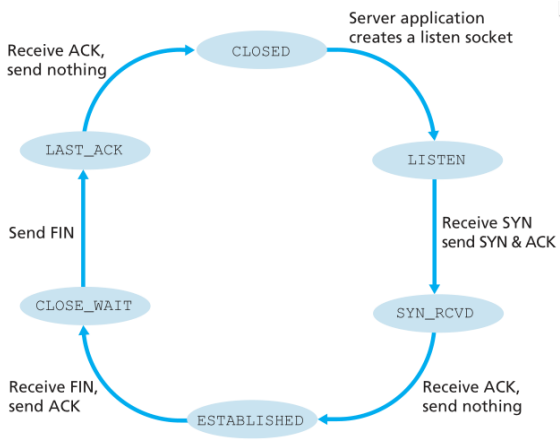

Figure 4: Connection FSM for TCP server

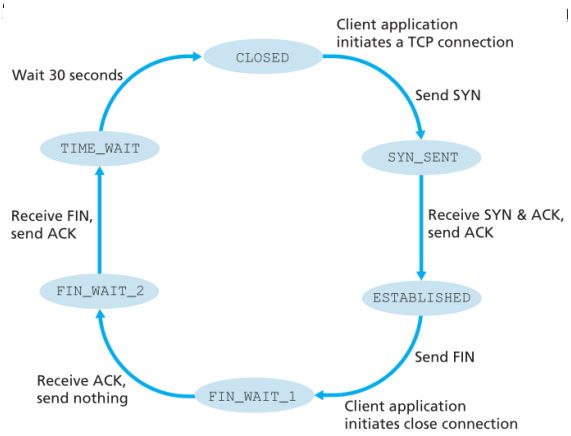

Figure 5: Connection FSM for TCP client

The deadline for this part is separate from the other tasks! That way we can be sure you have started work on the project and provide you with a correct state machine before you get too far into implementing.

4.2 Developing your implementation

Although Python can be written just fine in a text editor, consider using the support an integrated development environment provides. One option is PyCharm, an IDE that provides, among other things, code analysis, integration with version control systems, and a graphical debugger. Free educational licenses are available.

Once you are done reading this document, **read the code we have provided!** We have added a lot of documentation that hopefully clarifies the details for each individual method.

You are strongly encouraged to develop your bTCP implementation in an incremental manner. **As mentioned before, correctly handshaking is actually one of the more complicated things about this protocol. Considering the distribution of points, you may want to, and are in fact encouraged to *postpone* implementing the handshake until last.**

Instead of negotiating with a handshake, simply hardcode some sensible values. Make `connect` and `accept` move into the right state without sending anything, and start the server *before* the client.

With that setup, make your implementation work under ideal conditions and introduce other conditions incrementally:

1. ideal network (no packet loss, bit flips, etc).
2. network with spurious bit flips.
3. network with duplicate packets.
4. network with reordered packets.

5. network with packet loss.

The unit tests were created in order to help you with the aforementioned incremental approach. When you run it, it will start by testing the ideal network, then a network with bit flips, etc. See 4.4 for details on testing.

4.3 Debugging

4.3.1 Logging (a.k.a `print` debugging on steroids)

Pay particular attention to the use of the `logging` module, and the `logger` object in each file. The `logger` allows you to log information to standard error output (or a file). It outputs which module (i.e. file) is being logged from, as well as the time. It also allows you to cumulatively turn log levels on or off, so that you don't *always* see **all** debug logging. The provided client and server applications take a commandline argument `-l`, or `--loglevel`, for that. It defaults to `DEBUG`. Use `--help` to see other options. Note that writing text to the terminal is an operation that slows the application down, so play around with less verbose log levels if that is a problem.

Read at least the first two sections ("When to use logging" and "A simple example") of https://docs.python.org/3/howto/logging.html. Also read "logging variable data" from https://docs.python.org/3/howto/logging.html#logging-variable-data.

Proper logging is much more useful than sprinkling `print()` all over your code. While developing, it's probably a good idea to consistently add `logger.warning()`, `logger.error()`, and `logger.critical()` where appropriate. But even more importantly, consistently add calls to `logger.info()` and `logger.debug()` for more detailed information (like intermediate printing of variable values, or function entry and exit points). This way, all you have to do is run the programs with `--loglevel=DEBUG` to get a lot of insight in what is happening, rather than having to add print statements until you find your bug.

4.3.2 Wireshark bTCP dissector

Ciske has helpfully written a dissector for Wireshark, allowing you to inspect the bTCP header values on actual traffic captures.

It's written in Lua, and provided as a separate file. To use it, you will need to copy it to a Wireshark "Lua Plugins" directory. You can find this by opening Wireshark and navigating to

```
Help -> About Wireshark -> Folders
```

Bear in mind that the Personal Lua Plugins directory may change depending on what user you run Wireshark as (in particular, root or not). After copying the file, restart Wireshark. If you run a capture on the loopback interface `lo` while generating bTCP traffic, you should see it immediately dissected as bTCP.

4.4 Testing your implementation

We have provided two complementary ways to test your code: file transfer application and unit tests. Initially, you don't need to run the unit tests — get the file transfer applications to run without crashing first.

4.4.1 File transfer on top of the bTCP protocol

We have provided a pair of *very* simple python applications that use the bTCP transport layer:

- one server creating a bTCP server socket, accepting a connection, and receiving (a lot of) data; and
- one client creating a bTCP client socket, connecting to the server, sending (a lot of) data, and shutting down the connection.

The easiest way to test your code under ideal conditions is by seeing if these applications can run and communicate without crashing, and that the data has not been altered for the recipient.

Our initial implementation simply reads `large_input.py`, an 128 MiB file, from the filesystem and transfers its contents. This is enough to properly test your implementation. You can tell it to use a different (smaller) file through command line arguments. Run `./client_app.py -h` for details.

To compare an input and output file, you can use the `cmp` utility on the command line. It will tell you exactly where the first difference arises.

```
cmp large_input.py output.file
large_input.py output.file differ: byte 153, line 1
```

tells me that after 152 bytes, for some reason a difference occurred.

On the other hand,

```
cmp large_input.py output.file
cmp: EOF on output.file after byte 2016, in line 1
```

tells me that `output.file` is only 2016 bytes long (so probably only two segments were accepted by the other side), *but* all the data up to there equals the data in the input file.

Finally, if

```
cmp large_input.py output.file
```

doesn't seem to give you *any* output at all, that means the files have the exact same contents.

Looking beyond the first difference may be useful to diagnose e.g. a data stream that is missing dropped segments. In that case, you can try using `diff` in addition to `cmp`.

4.4.2 Unit tests

Once the file transfer applications can run without crashing due to `NotImplementedErrors`, you can run the unit tests by running `python3 unittests.py -v` or `python3 -m unittests -v`. Note that if you don't pass the `-v` switch, the unit tests do not tell you which tests succeed, only which ones fail. You can also add the `-f` switch to "fail fast", i.e. stop testing at the first failure, so that you won't be drowned in error messages. If you want to work on one specific test, you can run only that one using the `-k` flag. For example, `python3 unittests.py -k test_11` will run only `test_11_hello_world`.

You'll see that the provided code already passes some of the tests when the `NotImplementedErrors` are removed. This is due to how the provided implementation already implements UDP-like behaviour, and localhost is (generally) a perfect network. Once you actually try implementing the handshake or reliable data transfer, those tests may start failing until your implementation works.

The unit tests not only help you develop your bTCP implementation, but also to a large extent determine your grade; we refer you back to 1 for the details.

We might release new unit tests over the course of the project, so keep an eye on Brightspace for updates. (The list of tests that determines your grade will, of course, not change.)

4.4.3 Final words on testing

If our implementation of the file transfer application and our unit tests are insufficient for your needs, you can add your own. In that case, you do not have to send an actual *file* from the file system through bTCP, you can just operate on `bytes` objects within Python. As an example, `large_input.py` is also a python source file that contains a `bytes` object containing 85 MiB of data. Similarly, `small_input.py` has a `bytes` object containing 72KiB of data. You can use those objects directly if you import them.

Regardless of what you do, make sure you test your code under ideal network conditions with data larger than 64 MiB. We'll let you figure out yourself why we insist on that.

Additionally, bear in mind that *our* tests need to succeed to determine your grade.

5 Deliverables

To summarize, you are supposed to hand in the following:

1. *Early deadline!* Drawings of the bTCP connection management finite state machines. Submit these in the "bTCP Finite State Machines" Assignment on Brightspace.

2. *Early deadline!* Output of succesfully running the three basic tests without failures, specifically: `python3 unittests.py -l DEBUG -v -k test_00 -k test_10 -k test_11` Submit this in the "bTCP basic tests" Assignment on Brightspace.

3. Your Python 3 implementation of the bTCP transport layer and a *small* report about it. Submit these in the "bTCP implementation" Assignment on Brightspace.

   - Your code should work on the standard CPython implementation and use only the standard library (so no external dependencies). We'd prefer if you'd use Python 3.14, but we'll accept any Python 3.X code where $10 \leq X \leq 14$. Please make clear in your submission on which Python version(s) you've tested your code.

     As part of this, especially if your code does not work with the versions provided by us:

     – A *simple* file transfer application using your bTCP transport layer to transfer a (large-ish) file (or hardcoded blob of data; you don't *have* to read this from the filesystem). We have already provided a working one, but you should alter it if needed for your bTCP implementation.

   - A small report documenting your design decisions and justifications for these decisions. You should also document which parts work, which parts fail, and which parts you simply did not get around to implementing yet. Base this loosely on the point distribution given at the start of this project description.

For instructions on how to submit these deliverables and their deadlines, please read pages 1, 2 and 3 of this document carefully.